



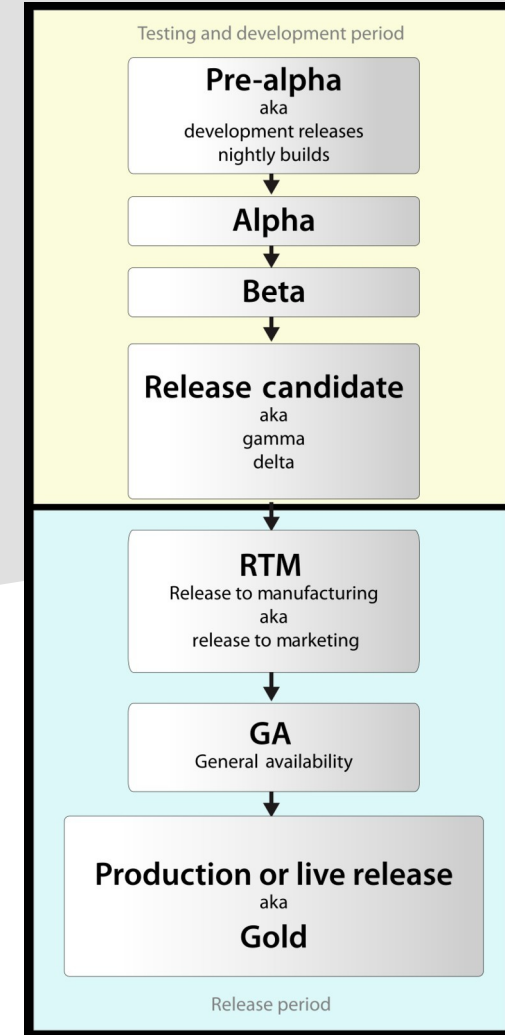
1.- Introducción

Cualquier desarrollo software debería pasar por varias etapas antes de llegar a su estado final (producción / cliente).

Cada etapa debe recibir **feedback** del usuario / cliente, fomentando así las metodologías ágiles.

La fase **Beta** puede considerarse un Producto Mínimo Viable (**PMV**) del software.

En la fase **Beta** pueden intervenir usuarios finales a los que se ofrece el software para que lo prueben y detecten errores. Es más fácil y barato, pues no tiene apenas coste y realizan las pruebas miles de personas.





1.- Introducción

Del diseño y realización de pruebas se encargan los departamentos:

- **QA** (Quality Assurance): interviene en todo el desarrollo.
- **QC** (Quality Control): interviene en la finalización del proyecto.

Se realizan las suficientes pruebas exhaustivas para convencer al desarrollador y al cliente de que el programa funciona correctamente.

No merece el coste probar todas las posibilidades de un programa a no ser que sea un programa de VITAL importancia.

De este modo la probabilidad de errores en la fase de producción disminuye notablemente.





1.- Introducción

Diferencias entre QA y QC

QA (Quality Assurance)	QC (Quality Control)
Diseñan y definen todos los parámetros de aceptación de un paquete de software.	Controlan el comportamiento del producto final.
Trabajan junto a desarrolladores, ingenieros, managers y el cliente.	Trabajan junto a QA.
Están presentes desde la fase de diseño del producto.	Entran en acción cuando el producto está finalizado.
Orientado al proceso.	Orientado al producto.
Diseñan y ejecutan antes de tener un producto finalizado.	Ejecutan durante la puesta en pre-producción.
Pruebas a nivel de código fuente (White box).	Pruebas a nivel de servicio (Black box).



2.- Procedimientos de pruebas

Definen **qué** va a probarse, **como** va a probarse, **quien** lo va a probar y bajo que circunstancias.

Los objetivos pueden ser:

- Detectar errores.
- Comprobar rendimiento.
- Visualizar interfaz.
- ...

Por tanto, la ausencia de errores no implica que la prueba se supere satisfactoriamente.





2.- Procedimientos de pruebas

Se recomienda que las realicen equipos especializados. (QA y/o QC)

Un plan de pruebas incluye:



1. **Introducción:** objetivo, responsables, planificación, duración.
2. **Partes o Módulos** a probar.
3. **Características** que se probarán (pueden no ser todas).
4. **Criterios** de validez.
5. **Requerimientos** del entorno (software, hardware, comunicaciones, seguridad).
6. **Pasos** para ejecutar las pruebas.

*Los programadores realizan sus propias pruebas, pero no contemplan todos los casos cruciales, “**vician**” el proceso y no encuentran todos los posibles errores.*



2.- Procedimientos de pruebas

CASOS DE PRUEBA

Se crean con el objetivo de detectar fallos.

Cada caso de prueba busca y detecta un único error, por lo que es importante diseñar suficientes casos de prueba para comprobar no solo la funcionalidad del sistema.

RECUERDA: probar es ejecutar casos de prueba uno a uno, pero que un software pase todos los casos de prueba no significa que el programa no contenga errores.

No hay que probar 1000 veces lo mismo, se trata de diseñar distintos tipos de pruebas.



2.- Procedimientos de pruebas

CASOS DE PRUEBA

[Identificador] 01 - [Título] Caso prueba ejemplo QC (Quality Control)	
Objetivo	Comprobar que un usuario correcto entra al sistema y la fecha y la hora de entrada queda registrada.
Datos entrada	Usuario: Jorge Contraseña: demaximaseguridad
Pre-condición	Existe el usuario “Jorge” en la tabla Usuarios con la contraseña “demaximaseguridad” encriptada en MD5.
Resultado esperado	El usuario entra al sistema.
Post-condición	Se registra el usuario, la fecha y hora en la tabla Log.
Estado	[Éxito, Fallido, Pendiente ejecución, En construcción]
Comentarios	

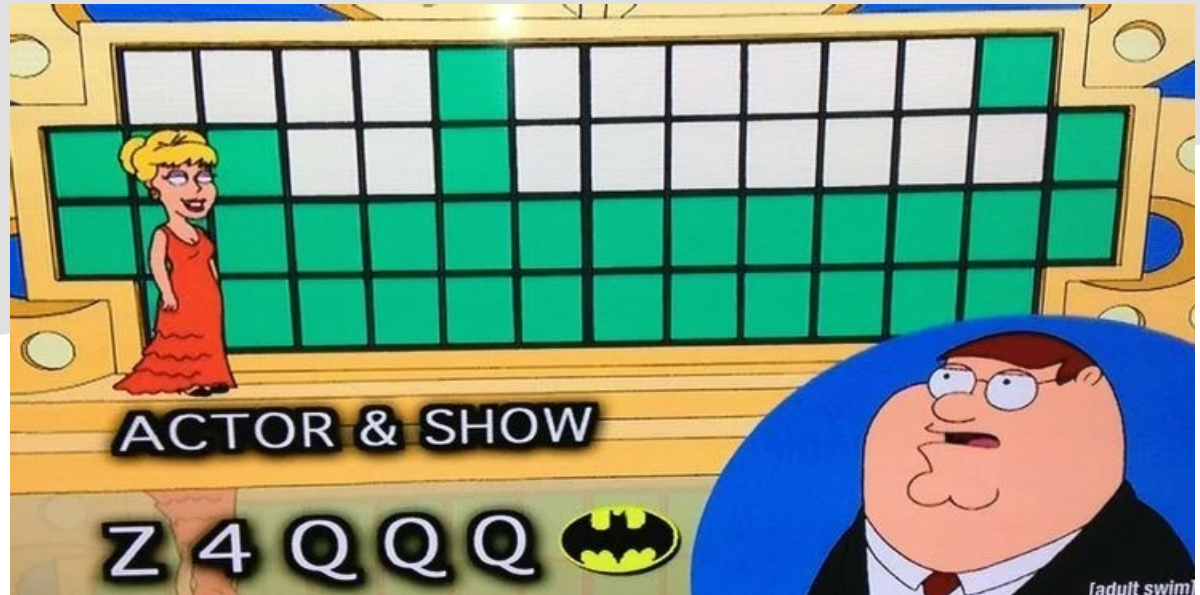


2.- Procedimientos de pruebas

CODIFICACIÓN DE PRUEBAS

La codificación de una prueba debe incluir conjuntos de datos (set):

- Validos.
- Inválidos.
- Fuera de rango.
- Disparatados.





2.- Procedimientos de pruebas

EJECUCIÓN DE PRUEBAS

Para la ejecución hay que preparar el entorno según las especificaciones de la prueba: hardware, software, comunicaciones, permisos...

Las pruebas deben ejecutarse una a una.

Si se detecta un error hay que **documentarlo**: prueba ejecutada, datos utilizados, error detectado (puede ser un resultado incorrecto, un fallo del sistema, un problema de conexión...)

Errores aleatorios: cuando un error no se replica todas las veces con los mismos datos, se dice **no reproducible**.





3.- Tipos de pruebas

Pruebas de caja blanca

Analizan el código, cómo se hace.
Ejemplo: pruebas estructurales, de arquitectura, reusabilidad...

Pruebas de caja negra

No evalúan como se hace, solo lo que hace. Ejemplo: pruebas funcionales, compatibilidad, rendimiento...

Pruebas regresivas

Cuando se realiza un cambio, hay que comprobar que no se ha estropeado nada que ya funcionara.





4.- Pruebas de caja blanca

Su objetivo es probar el código. Las personas que realizan las pruebas están en contacto con el código fuente.

Existen 3 tipos de pruebas de **caja blanca**:

- Pruebas de cubrimiento.
- Pruebas de condiciones.
- Pruebas de bucles.



Se dice que en una prueba de caja blanca tenemos un conocimiento completo. Está relacionada con el programador y la fase de desarrollo.



4.- Pruebas de caja blanca

PRUEBAS DE CUBRIMIENTO

Objetivo: ejecutar, al menos una vez, todas las líneas de código del programa.
Habrá que generar suficientes datos de prueba para cubrir todos los “caminos”.
Pueden existir condiciones imposibles que no lo permitan.

```
if (edad < 18) {  
    // Eres menor de edad,  
    // NO puedes salir del instituto  
} else {  
    // Eres mayor de edad  
    // SI puedes salir del instituto  
}
```

```
if (edad > 20 && edad < 10) {  
    // Es imposible cumplir la condición  
    // Nunca se ejecutará este bloque  
}
```





4.- Pruebas de caja blanca

PRUEBAS DE CONDICIONES

Objetivo: probar todas las condiciones posibles y comprobar los resultados.

Pueden estar incluidas en una prueba de cubrimiento.

```
if (starWars == true && lotR == true && harryPotter == true) {  
    friki = true;  
}
```



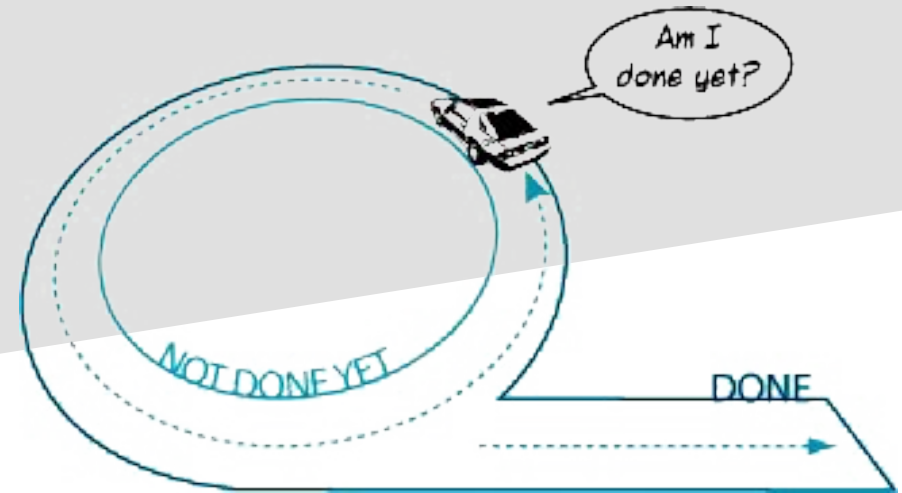


4.- Pruebas de caja blanca

PRUEBAS DE BUCLES

Objetivo: probar un bucle un número especial de veces.

- Repetir el máximo.
- Repetir el máximo + 1.
- Repetir el máximo - 1.
- Repetir 0.
- Repetir 1.
- Repetir N veces ($N > 0 \ \&\& \ N < \text{máximo}$)



Hay que prestar especial atención a los bucles anidados.



5.- Pruebas de caja negra

Su objetivo es probar la funcionalidad de la aplicación / programa.

Existen 3 tipos de pruebas de **caja negra**:

- Pruebas de clases de equivalencia de datos.
- Pruebas de valores límite.
- Pruebas de interfaces.



La persona que realiza las pruebas usará el programa igual que lo haría un usuario final. Está relacionado con la fase de pruebas.



5.- Pruebas de caja negra

PRUEBAS DE CLASES DE EQUIVALENCIA DE DATOS

Objetivo: probar todas las clases de datos válidos e inválidos.

Ejemplo:

Probar un campo contraseña de 6 a 12 caracteres alfanuméricos, que incluya al menos una letra mayúscula, una minúscula y un número.

- *Clases válidas: “mAXIMA0”, “Seguridad1” y “MiPerroR3x”.*
- *Clases inválidas: “sandwich”, “deJamonyork” y “ques0”.*

Al probar un caso de prueba con más de un campo (por ejemplo usuario y contraseña), se probarán individualmente las clases inválidas para evitar que los errores se “enmascaren”.



5.- Pruebas de caja negra

PRUEBAS DE VALORES LÍMITE

Objetivo: generar valores que puedan probar que la interfaz y el programa funcionan correctamente. Centrándose en los límites.

Ejemplo:

En la web de un banco, probar el campo “cantidad” de las transferencias (limitadas a 10.000€)

- *Valores en rango (de 1 a 10.000 €).*
- *Valores fuera de rango (-100, 0, 10.001, 2.000.000 €).*
- *Valores no válidos (en rango pero sin saldo suficiente).*





5.- Pruebas de caja negra

PRUEBAS DE INTERFACES

Objetivo: probar la interfaz de usuario (**GUI**) del programa.

Se probará siguiendo el manual de usuario, como si fuera un usuario final.

Se deben testear 2 aspectos.

- **Usabilidad:** el software debe ser fácil de usar, eficiente, homogéneo e intuitivo. Se deben tener en cuenta: atajos de teclado, autocompletar, valores por defecto... El feedback del usuario final es indispensable.
- **Accesibilidad:** el software debe estar diseñado para que pueda ser usado por otras máquinas, usuarios con discapacidad o en condiciones desfavorables (ausencia de teclado, ratón, pantalla...) .



6.- Planificación de pruebas

Hay que planificar qué tipo de pruebas van a realizarse y cuando.

- Proyectos pequeños: se encargarán los propios desarrolladores.
- Proyectos grandes: se encargará el departamento de QA.

La planificación y ejecución de pruebas lleva asociada una documentación.

Es muy útil conservarla para su reutilización:

- Herramientas o sistemas utilizados.
- Programas y módulos de prueba.
- Datos utilizados (sets).
- Resultados y comentarios.





6.- Planificación de pruebas

PRUEBAS UNITARIAS

Se realizan en las primeras fases de diseño y desarrollo, aislando partes “indivisibles” de código para su testeo. (Desarrolladores, **caja blanca**).

En la POO deben realizarse a nivel de objeto y luego a nivel paquete o librería.

PRUEBAS DE ACEPTACIÓN O VALIDACIÓN

Verifican todos los requisitos del programa (Tester, **caja negra**).

También prueban el rendimiento (pruebas de carga y estrés).

El estado del proyecto en este punto es **alfa** o **beta**.



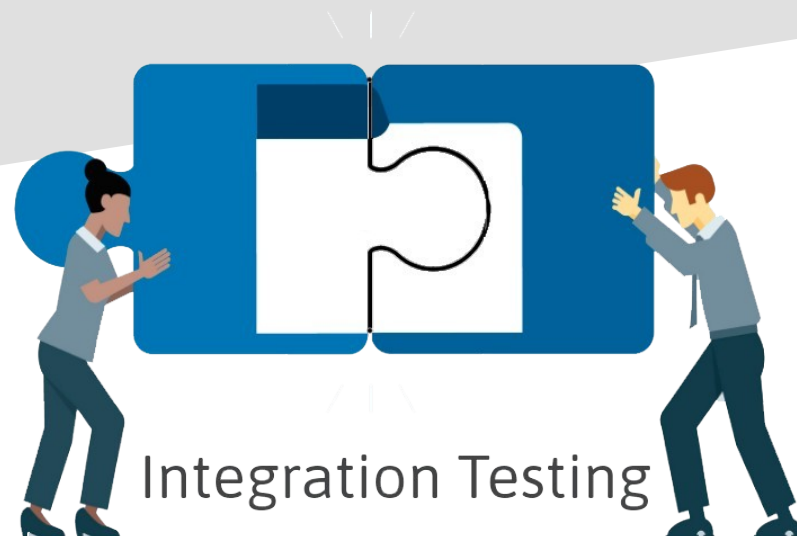
6.- Planificación de pruebas

PRUEBAS DE INTEGRACIÓN

Se realizan al final de la fase de diseño y codificación, una vez realizadas y aprobadas las pruebas unitarias.

Se centran principalmente en probar la comunicación entre los componentes y sus comunicaciones.

Las pruebas de integración comprueban que todas las piezas encajan.





7.- Calidad del software

La calidad del software es un aspecto fundamental para el éxito o fracaso.

Seguir un proceso de desarrollo software es indispensable para garantizar la calidad del producto final.

Criterios para evaluar la calidad del software (benchmark):

- **Pruebas de carga.** (Baja, media y alta; tiempos de respuesta)
- **Pruebas de estrés.** (Encontrar límite de carga de trabajo)
- **Pruebas de estabilidad.** (Alternar picos de carga breves y duraderos)
- Número de errores por línea de código.

Crisis de los 90, crisis actual, programadores creadores VS usuarios



8.- Herramientas de depuración de código

La elaboración y ejecución de pruebas puede ser un trabajo tedioso.

Pueden usarse herramientas / frameworks que facilitan y automatizan este proceso.

Para trabajar con Java, podemos usar **JUnit**.

Aquí podemos ver un video de cómo crear pruebas con JUnit